

Python JSON Module

```
# =====
# JSON MODULE - COMPLETE PYTHON SCRIPT
# =====

import json

# 1. PYTHON DICTIONARY → JSON STRING

customer = {
    "id": 1,
    "name": "Ahmed",
    "age": 25,
    "city": "Cairo"
}

json_string = json.dumps(customer)

print("Python Dictionary:")
print(customer)

print("\nJSON String:")
print(json_string)

print("Type:", type(json_string))

# 2. JSON STRING → PYTHON DICTIONARY
json_data = '{"id": 2, "name": "Mohamed", "age": 30}'

customer = json.loads(json_data)

print("\nJSON String:")
print(json_data)

print("\nPython Dictionary:")
print(customer)

print("Type:", type(customer))

# 3. ACCESS JSON DATA
print("\nCustomer ID:", customer["id"])
print("Customer Name:", customer["name"])
print("Customer Age:", customer["age"])

# 4. PRETTY JSON
customer = {
    "id": 1,
    "name": "Ahmed",
    "skills": ["Python", "SQL", "Spark"],
    "address": {
        "city": "Cairo",
        "country": "Egypt"
    }
}

pretty_json = json.dumps(
    customer,
    indent=4
)

print("\nPretty JSON:")
print(pretty_json)
```

```

# =====
# 5. SORT JSON KEYS
# =====

sorted_json = json.dumps(
    customer,
    indent=4,
    sort_keys=True
)

print("\nSorted JSON:")
print(sorted_json)

# =====
# 6. CONVERT PYTHON LIST → JSON
# =====

customers = [
    {
        "id": 1,
        "name": "Ahmed"
    },
    {
        "id": 2,
        "name": "Mohamed"
    },
    {
        "id": 3,
        "name": "Omar"
    }
]

json_customers = json.dumps(
    customers,
    indent=4
)

print("\nCustomers:")
print(json_customers)

# =====
# 7. WRITE JSON TO A FILE
# =====

data = {
    "pipeline": "customer_etl",
    "status": "success",
    "records_processed": 1000
}

with open("pipeline.json", "w") as file:
    json.dump(
        data,
        file,
        indent=4
    )

print("\nJSON file created: pipeline.json")

```

```

# =====
# 8. READ JSON FROM A FILE
# =====

with open("pipeline.json", "r") as file:
    data = json.load(file)

print("\nData loaded from JSON file:")
print(data)

print("Pipeline:", data["pipeline"])
print("Status:", data["status"])
print("Records:", data["records_processed"])

# =====
# 9. DUMP vs DUMPS
# =====

# dumps()
# Python object → JSON STRING

data = {
    "name": "Ahmed",
    "age": 25
}

json_string = json.dumps(data)

print("\nDUMPS:")
print(json_string)
print(type(json_string))

# dump()
# Python object → JSON FILE

with open("customer.json", "w") as file:
    json.dump(data, file, indent=4)

print("\nDUMP:")
print("Data written to customer.json")

# =====
# 10. LOAD vs LOADS
# =====

# loads()
# JSON STRING → Python object

json_string = '{"name": "Ahmed", "age": 25}'

data = json.loads(json_string)

print("\nLOADS:")
print(data)
print(type(data))

# load()
# JSON FILE → Python object

with open("customer.json", "r") as file:
    data = json.load(file)

print("\nLOAD:")
print(data)
print(type(data))

```

```
# =====  
# 11. JSON DATA TYPES  
# =====
```

```
data = {  
    "name": "Ahmed",      # string  
    "age": 25,            # integer  
    "salary": 15000.50,   # float  
    "is_active": True,    # boolean  
    "skills": ["Python", "SQL"], # list  
    "address": {          # dictionary  
        "city": "Cairo"  
    },  
    "phone": None         # None  
}
```

```
json_data = json.dumps(  
    data,  
    indent=4  
)
```

```
print("\nJSON Data Types:")  
print(json_data)
```

```
# =====  
# 12. JSON → PYTHON TYPE MAPPING  
# =====
```

```
"""  
JSON                Python  
  
object              →    dict  
array               →    list  
string              →    str  
number              →    int / float  
true                →    True  
false               →    False  
null                →    None  
"""
```

```
# =====  
# 13. HANDLE INVALID JSON  
# =====
```

```
invalid_json = '{"name": "Ahmed", "age": 25'
```

```
try:  
    data = json.loads(invalid_json)  
    print(data)
```

```
except json.JSONDecodeError:  
    print("\nInvalid JSON!")
```

```

# =====
# 14. DATA ENGINEERING EXAMPLE - API RESPONSE
# =====

api_response = """
{
    "status": "success",
    "data": [
        {
            "id": 1,
            "name": "Laptop",
            "price": 25000
        },
        {
            "id": 2,
            "name": "Phone",
            "price": 15000
        },
        {
            "id": 3,
            "name": "Tablet",
            "price": 10000
        }
    ]
}
"""

# JSON string → Python dictionary
response = json.loads(api_response)

print("\nAPI Status:")
print(response["status"])

products = response["data"]

print("\nProducts:")

for product in products:
    print(
        product["id"],
        product["name"],
        product["price"]
    )

# =====
# 15. DATA ENGINEERING - TRANSFORM JSON DATA
# =====

products = response["data"]

for product in products:
    product["category"] = "Electronics"

print("\nTransformed Products:")

print(
    json.dumps(
        products,
        indent=4
    )
)

```

```

# =====
# 16. DATA ENGINEERING - SIMPLE ETL
# =====

# EXTRACT

json_data = """
[
    {"id": 1, "name": "Laptop", "price": 25000},
    {"id": 2, "name": "Phone", "price": 15000},
    {"id": 3, "name": "Tablet", "price": -500}
]
"""

products = json.loads(json_data)

# TRANSFORM + VALIDATE

valid_products = []

for product in products:
    if product["price"] >= 0:
        product["category"] = "Electronics"
        valid_products.append(product)

# LOAD

with open("products_clean.json", "w") as file:
    json.dump(
        valid_products,
        file,
        indent=4
    )

print("\nETL completed!")

print("Valid records:", len(valid_products))

print(
    json.dumps(
        valid_products,
        indent=4
    )
)

# =====
# 17. MOST IMPORTANT FUNCTIONS
# =====

print("""
=====
JSON FUNCTIONS
=====

json.dumps()
    Python object → JSON string

json.loads()
    JSON string → Python object

json.dump()

```

Python object → JSON file

`json.load()`

JSON file → Python object

=====
REMEMBER
=====

`dumps` → string
`loads` → string

`dump` → file
`load` → file

=====
DATA ENGINEERING
=====

API JSON

↓

`json.loads()`

↓

Python Dictionary/List

↓

Transform

↓

`json.dump()`

↓

JSON File

""")